

```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.P04.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S04_Main";
    }
    /// <summary>
    /// S00_Boot      Mock → →      T001④
    ///      Boot      UI T001b Boot      =
    /// </summary>
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null && !op.isDone) yield return null;
        }
    }
}

// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.P04.Boot
{
    /// <summary>
```

```

/// Boot      T001
/// ①Logger→② →③Config→④Save→⑤ILocale→⑥Ads.Preload→⑦      [BOOT] n/7
/// ⑦      Play =SceneManager.LoadSceneAsync Editor dry-run=EditorSceneManager.OpenScene
/// X006      SY MOCK
/// &lt;/summary&gt;
public sealed class BootSequence
{
    public const int TotalSteps = 7;
    readonly Func&lt;bool&gt; _switchScene;
    readonly List&lt;string&gt; _lines = new();
    public BootSequence(Func&lt;bool&gt; switchScene)
    {
        _switchScene = switchScene;
    }
    public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
    public bool RunAll()
    {
        bool ok = true;
        ok &= Run(1, "Logger", LoggerStep);
        ok &= Run(2, "Time", TimeStep);
        ok &= Run(3, "Config", ConfigStep);
        ok &= Run(4, "Save", SaveStep);
        ok &= Run(5, "ILocale", LocaleStep);
        ok &= Run(6, "Ads.Preload", AdsStep);
        ok &= Run(7, "SwitchMain", () =&gt; _switchScene != null && _switchScene());
        return ok;
    }
    bool Run(int n, string name, Func&lt;bool&gt; step)
    {
        bool result;
        try
        {
            result = step();
        }
        catch (Exception e)
        {
            Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
            result = false;
        }
        string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
        _lines.Add(line);
        Logger.Info("BOOT", line);
        return result;
    }
    static bool LoggerStep() =&gt; true; //      JS sink platform T001c
    static bool TimeStep()
    {
        var time = ServiceLocator.Get&lt;ITimeService&gt;();
        return time.ServerNowMs() &gt; 0 && !time.TimeCheated();
    }
}

```

```
static bool ConfigStep()
{
    var config = ServiceLocator.Get<IConfigService>();
    var update = config.CheckUpdate("mock://manifest");
    return update.NewVersion != null; // mock= X009 manifest
}
static bool SaveStep()
{
    var save = ServiceLocator.Get<ISaveService>();
    save.Save("boot_probe", "{}");
    return !string.IsNullOrEmpty(save.Load("boot_probe"));
}
static bool LocaleStep()
{
    var locale = ServiceLocator.Get<ILocaleService>();
    return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont); // +
}
static bool AdsStep()
{
    ServiceLocator.Get<IAdsService>().Preload("boot_warmup");
    return true;
}
}
}
// BootServices.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.P04.Boot
{
    /// <summary>
    /// Mock SY MOCK X006 Mock X002a AppID touristappid
    /// T001c WX/TT Channel Editor Boot
    /// </summary>
    public static class BootServices
    {
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet<ITimeService>(out _)) return; //
            ServiceLocator.Register<ITimeService>(new MockTimeService());
            ServiceLocator.Register<IConfigService>(new MockConfigService());
            ServiceLocator.Register<ISaveService>(new MockSaveService());
            ServiceLocator.Register<ILocaleService>(new MockLocaleService());
            ServiceLocator.Register<IAdsService>(new MockAdsService());
            Logger.Info("BOOT", "mock services registered (SY MOCK)");
        }
    }
}
```

```

}
public sealed class MockTimeService : ITimeService
{
    public TimeSource Source => TimeSource.Mock;
    public long ClientNowMs() => DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// <summary>SY MOCK X006 TCB serverTime→HTTPS Date → T001b </summary>
    public long ServerNowMs() => ClientNowMs();
    public bool TimeCheated() => Math.Abs(ServerNowMs() - ClientNowMs()) > 120_000;
}
public sealed class MockConfigService : IConfigService
{
    public T GetTable<T>(string tableId) where T : class
        => throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable<T>(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        => new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) => "mock-0.0.1";
}
public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) => PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) => PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// <summary>X006 T001b v1→v2 </summary>
    public SyncResult CloudSync() => new SyncResult(false, null);
}
public sealed class MockLocaleService : ILocaleService
{
    public void SetLang(string lang) { }
    /// <summary>key key T001b cfg_i18n + NUnit </summary>
    public string GetText(string key, params string[] args)
    {
        Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
        return key;
    }
    public FontSet GetFontSet()
        => new FontSet("LegacyRuntime.ttf", "LegacyRuntime.ttf", "LegacyRuntime.ttf");
}
public sealed class MockAdsService : IAdsService
{
    public event Action<AdLifecycle> OnAdEvent;
    public Task<RewardResult> ShowRewarded(string pointId)
        => Task.FromResult(new RewardResult { Completed = true, Reason = "mock" });
    public void ShowInterstitial(string pointId) { }
    public void ShowBanner(string pointId) { }
}

```

```

        public void Preload(string pointId)
        {
            Logger.Info("ADS", "preload (mock): " + pointId);
            OnAdEvent?.Invoke(new AdLifecycle(AdLifecycleType.Fill, pointId, "mock"));
        }
    }
}

// MainUI.cs
using UnityEngine;
using UnityEngine.SceneManagement;
namespace Sy.P04.Newbie
{
    /// <summary>S04          →S04_Battle M5          —          </summary>
    public sealed class MainUI : MonoBehaviour
    {
        public void StartHunt()
        {
            SceneManager.LoadScene(NewbieSceneIds.Battle);
        }
        void Awake()
        {
            Application.targetFrameRate = 30;
            Sy.Core.MainArtInjector.Apply(transform, "P04_scene_01", "P04_unit_01");
        }
    }
}

// BattleLauncher.cs
using System.Collections.Generic;
using UnityEngine;
namespace Sy.P04.Newbie
{
    public static class NewbieSceneIds
    {
        {
            public const string Main = "S04_Main";
            public const string Battle = "S04_Battle";
        }
        /// <summary>
        /// S04_Battle      IdleSim+TutorialFlow+BattleView  Tick
        ///   Sy.Telemetry Editor =Debug sink T001e          +TCB
        /// HUD              6
        /// </summary>
        public sealed class BattleLauncher : MonoBehaviour
        {
            IdleSim _sim;
            TutorialFlow _tutorial;
            BattleView _view;
            Sy.Telemetry.TelemetryService _telemetry;
            int _lastFloor = -1, _lastCharms = -1, _lastKills = -1, _lastCodex = -1;
            void Awake()
            {

```

```
Application.targetFrameRate = 30;
var canvas = FindObjectOfType<Canvas>();
_sim = new IdleSim();
_view = gameObject.AddComponent<BattleView>();
_view.Initialize(canvas, _sim);
_telemetry = new Sy.Telemetry.TelemetryService();
_telemetry.AddSink(new TelemetryDebugSink());
_tutorial = new TutorialFlow(
    _sim,
    hint => {
        Debug.Log("[HINT] " + hint);
        _view.SetHint(hint);
    },
    step => { _telemetry.Track(Sy.Telemetry.EventIds.NewbieStep,
        new Dictionary<string, string> { { "step", step.ToString() } }));
    _view.SealRequested += OnSealRequested;
    _view.FormationPlaced += OnFormationPlaced;
    _sim.StateChanged += OnStateChanged;
}
void Update()
{
    _sim.Tick(Time.deltaTime);
    _view.SyncView(_sim);
    if (_sim.Floor != _lastFloor) { _lastFloor = _sim.Floor; _view.SetFloor(_lastFloor); }
    if (_sim.SealCharms != _lastCharms) { _lastCharms = _sim.SealCharms; _view.SetCharms(_lastCharms); }
    if (_sim.Kills != _lastKills) { _lastKills = _sim.Kills; _view.SetKills(_lastKills); }
    if (_sim.CodexLitCount != _lastCodex) { _lastCodex = _sim.CodexLitCount; _view.SetCodex(_lastCodex); }
}
void OnSealRequested()
{
    _sim.PressSeal();
}
void OnFormationPlaced()
{
    _sim.PlaceGudiao();
}
void OnStateChanged(IdleSim.IdleState state)
{
    switch (state)
    {
        case IdleSim.IdleState.Fighting:
            _view.HideIntro();
            break;
        case IdleSim.IdleState.SealAnim:
            _view.PlaySealAnim();
            break;
        case IdleSim.IdleState.SealFailAnim:
            _view.PlaySealFail();
            break;
    }
}
```

```

        case IdleSim.IdleState.FloorSkip:
            _view.ShowFloorSkip(true);
            break;
        case IdleSim.IdleState.WallDemo:
            _view.ShowFloorSkip(false);
            _view.ShowWallBanner(true);
            break;
        case IdleSim.IdleState.Formation:
            _view.ShowWallBanner(false);
            _view.ShowFormationPanel(true);
            break;
        case IdleSim.IdleState.BossFight:
            _view.ShowFormationPanel(false);
            break;
        case IdleSim.IdleState.Settled:
            _view.ShowSettle(_sim.Kills, _sim.SealCharms, _sim.CodexLitCount);
            break;
    }
}

sealed class TelemetryDebugSink : Sy.Telemetry.TelemetryService.ISink
{
    public void Write(string evtId, Dictionary<string, string> fields)
    {
        var sb = new System.Text.StringBuilder(evtId);
        foreach (var kv in fields)
        {
            sb.Append(' ').Append(kv.Key).Append('=').Append(kv.Value);
        }
        Debug.Log("[TELEMETRY] " + sb);
    }
}

}

// BattleView.cs
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
namespace Sy.P04.Newbie
{
    /// <summary>
    ///         UGUI— / / / /
    /// X025         =         0.8s+         § V1.2         =         0.5s+
    ///         =         4-9         =
    ///     UI Initialize         YAML
    /// </summary>
    public sealed class BattleView : MonoBehaviour
    {
        //         x=-350         x=+500         0.3

```

```

const float FriendX = -350f;
const float FieldSpan = 850f;
static readonly float[] RowY = { 300f, 0f, -300f };
Canvas _canvas;
Sprite _square;
RectTransform _field;
readonly Image[] _friendHpFills = new Image[NewbieConfig.FriendCount];
Text _floorText, _charmText, _killsText, _hintText;
Button _sealButton;
Image _sealButtonImage;
Text _sealLabel;
RectTransform _introOverlay, _sealOverlay, _failOverlay, _skipBanner, _wallBanner, _formationPanel, _settlePanel
RectTransform _magicRing;      //      6
Image _sealFlash;              //
Text _settleStats;
readonly Image[] _codexSlots = new Image[NewbieConfig.CodexSlots];
readonly Text[] _codexLabels = new Text[NewbieConfig.CodexSlots];
readonly Dictionary<int, EnemyVisual> _enemies = new();
readonly List<int> _scratch = new();
public event Action SealRequested;
public event Action FormationPlaced;
static readonly Color PalRed = new Color(0.902f, 0.224f, 0.275f); //      #E63946
static readonly Color PalBlue = new Color(0.271f, 0.482f, 0.616f); //      #457B9D
static readonly Color PalGold = new Color(1f, 0.843f, 0f); //      #FFD700
static readonly Color PalGreen = new Color(0.165f, 0.616f, 0.561f); //      #2A9D8F
static readonly Color PalBlack = new Color(0.114f, 0.114f, 0.114f); //      #1D1D1D
static readonly Color PalFrostedGold = new Color(0.63f, 0.55f, 0.31f); //      .      $
static readonly Color PanelBg = new Color(0.07f, 0.08f, 0.10f, 0.95f);
public void Initialize(Canvas canvas, IdleSim sim)
{
    _canvas = canvas;
    _square = MakeSquareSprite();
    BuildBackdrop();
    BuildHud();
    BuildField();
    BuildCodex();
    BuildIntroOverlay();
    BuildSealOverlay();
    BuildFailOverlay();
    BuildSkipBanner();
    BuildWallBanner();
    BuildFormationPanel();
    BuildSettlePanel();
    SetHint("      .      .....");
}
//      _____
/// <summary>X027 generated art: fullscreen backdrop behind all UI; skipped
/// entirely when the art key is missing so headless checks stay green.</summary>
void BuildBackdrop()
{

```

```
var art = Sy.Core.ArtRuntime.Get("P04_scene_01");
if (art == null) return;
var go = new GameObject("ArtBackdrop", typeof(RectTransform), typeof(Image));
go.transform.SetParent(_canvas.transform, false);
var rt = (RectTransform)go.transform;
rt.anchorMin = Vector2.zero;
rt.anchorMax = Vector2.one;
rt.offsetMin = rt.offsetMax = Vector2.zero;
rt.SetAsFirstSibling();
var image = go.GetComponent<Image>();
image.sprite = art;
image.color = Color.white;
image.preserveAspect = true;
image.raycastTarget = false;
}
void BuildHud()
{
    var root = _canvas.transform;
    _floorText = MakeText(root, " 0 ", 46, new Vector2(0, 880), TextAnchor.UpperLeft, 300, 70);
    _floorText.color = PalGold;
    MakeText(root, "    " + NewbieConfig.StartLife, 46, new Vector2(0, 880), TextAnchor.UpperRight, 340, 70);
    _charmText = MakeText(root, "    x" + NewbieConfig.StartSealCharms, 44, new Vector2(0, 820), TextAnchor.UpperRight, 340, 70);
    _charmText.color = PalGold;
    _killsText = MakeText(root, " 0", 44, new Vector2(0, 820), TextAnchor.UpperRight, 400, 60);
    _hintText = MakeText(root, "", 40, new Vector2(0, -560), TextAnchor.MiddleCenter, 1020, 150);
    _hintText.color = new Color(0.95f, 0.87f, 0.6f);
    var btnGo = new GameObject("SealButton", typeof(Image), typeof(Button));
    btnGo.transform.SetParent(root, false);
    var btnRt = (RectTransform)btnGo.transform;
    btnRt.anchorMin = btnRt.anchorMax = new Vector2(0.5f, 0f);
    btnRt.anchoredPosition = new Vector2(0, 140);
    btnRt.sizeDelta = new Vector2(460, 140);
    _sealButtonImage = btnGo.GetComponent<Image>();
    _sealButtonImage.sprite = _square;
    _sealButtonImage.color = new Color(0.35f, 0.35f, 0.38f);
    _sealButton = btnGo.GetComponent<Button>();
    _sealButton.interactable = false;
    _sealLabel = MakeText(btnGo.transform, " 2 ", 46, Vector2.zero, TextAnchor.MiddleCenter, 460, 140);
    _sealLabel.color = new Color(0.7f, 0.7f, 0.7f);
    _sealButton.onClick.AddListener(() => SealRequested?.Invoke());
}
void BuildField()
{
    var fieldGo = new GameObject("Field", typeof(RectTransform), typeof(Image));
    fieldGo.transform.SetParent(_canvas.transform, false);
    _field = (RectTransform)fieldGo.transform;
    _field.anchorMin = _field.anchorMax = new Vector2(0.5f, 0.5f);
    _field.anchoredPosition = new Vector2(0, 60);
    _field.sizeDelta = new Vector2(1040, 1360);
    var fieldImg = fieldGo.GetComponent<Image>();
```

```

        fieldImg.sprite = _square;
        fieldImg.color = PanelBg;
        var lineGo = new GameObject("FriendLine", typeof(RectTransform), typeof(Image));
        lineGo.transform.SetParent(_field, false);
        var lineRt = (RectTransform)lineGo.transform;
        lineRt.anchorMin = lineRt.anchorMax = new Vector2(0.5f, 0.5f);
        lineRt.anchoredPosition = new Vector2(FriendX - 110f, 0);
        lineRt.sizeDelta = new Vector2(6, 1240);
        var lineImg = lineGo.GetComponent<Image>();
        lineImg.sprite = _square;
        lineImg.color = new Color(PalGold.r, PalGold.g, PalGold.b, 0.5f);
        for (int i = 0; i < NewbieConfig.FriendCount; i++)
        {
            var go = new GameObject("Friend_" + i, typeof(RectTransform), typeof(Image));
            go.transform.SetParent(_field, false);
            var rt = (RectTransform)go.transform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
            rt.anchoredPosition = new Vector2(FriendX, RowY[i]);
            rt.sizeDelta = new Vector2(150, 150);
            var image = go.GetComponent<Image>();
            image.sprite = _square;
            image.color = i == 0 ? PalGreen : i == 1 ? PalBlue : new Color(0.85f, 0.78f, 0.55f);
            MakeText(go.transform, NewbieConfig.FriendNames[i].Substring(0, 1), 52, new Vector2(0, -100), TextAnchor
            var hpGo = new GameObject("Hp", typeof(RectTransform), typeof(Image));
            hpGo.transform.SetParent(go.transform, false);
            var hpRt = (RectTransform)hpGo.transform;
            hpRt.anchorMin = new Vector2(0f, 1f);
            hpRt.anchorMax = new Vector2(1f, 1f);
            hpRt.anchoredPosition = new Vector2(0, -6);
            hpRt.sizeDelta = new Vector2(-8, 10);
            _friendHpFills[i] = hpGo.GetComponent<Image>();
            _friendHpFills[i].sprite = _square;
            _friendHpFills[i].type = Image.Type.Filled;
            _friendHpFills[i].fillMethod = Image.FillMethod.Horizontal;
            _friendHpFills[i].color = PalGreen;
        }
    }
    void BuildCodex()
    {
        var root = _canvas.transform;
        MakeText(root, " ", 36, new Vector2(400, 750), TextAnchor.UpperCenter, 200, 50).color = PalGold;
        for (int i = 0; i < NewbieConfig.CodexSlots; i++)
        {
            var go = new GameObject("Codex_" + i, typeof(RectTransform), typeof(Image));
            go.transform.SetParent(root, false);
            var rt = (RectTransform)go.transform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
            rt.anchoredPosition = new Vector2(400, 620 - i * 170);
            rt.sizeDelta = new Vector2(140, 140);
            var image = go.GetComponent<Image>();

```

```

        image.sprite = _square;
        image.color = new Color(0.16f, 0.16f, 0.18f);
        _codexSlots[i] = image;
        _codexLabels[i] = MakeText(go.transform, "?", 60, Vector2.zero, TextAnchor.MiddleCenter, 140, 140);
        _codexLabels[i].color = new Color(0.45f, 0.45f, 0.48f);
    }
}

void BuildIntroOverlay()
{
    _introOverlay = MakeOverlayPanel("IntroOverlay", new Color(0.03f, 0.03f, 0.05f, 0.92f));
    MakeText(_introOverlay, "      ", 90, new Vector2(0, 180), TextAnchor.MiddleCenter, 1000, 120).color = P
    MakeText(_introOverlay, "      —      ", 52, new Vector2(0, 40), TextAnchor.MiddleCenter, 1040, 90)
}

void BuildSealOverlay()
{
    _sealOverlay = MakeOverlayPanel("SealOverlay", new Color(0.05f, 0.04f, 0.02f, 0.85f));
    //      6      $ V1.2      = + 0.8s+
    var ringGo = new GameObject("MagicRing", typeof(RectTransform));
    ringGo.transform.SetParent(_sealOverlay, false);
    _magicRing = (RectTransform)ringGo.transform;
    _magicRing.anchorMin = _magicRing.anchorMax = new Vector2(0.5f, 0.5f);
    _magicRing.anchoredPosition = new Vector2(0, 120);
    _magicRing.sizeDelta = new Vector2(600, 600);
    Color[] ringColors = { PalRed, PalBlue, PalGold, PalGreen, PalRed, PalBlue };
    for (int i = 0; i < 6; i++)
    {
        var go = new GameObject("Shard_" + i, typeof(RectTransform), typeof(Image));
        go.transform.SetParent(_magicRing, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        float ang = i * 60f * Mathf.Deg2Rad;
        rt.anchoredPosition = new Vector2(Mathf.Cos(ang), Mathf.Sin(ang)) * 240f;
        rt.sizeDelta = new Vector2(70, 110);
        rt.localRotation = Quaternion.Euler(0f, 0f, i * 60f + 90f);
        var img = go.GetComponent<Image>();
        img.sprite = _square;
        img.color = ringColors[i];
    }
    var flashGo = new GameObject("Flash", typeof(RectTransform), typeof(Image));
    flashGo.transform.SetParent(_sealOverlay, false);
    var flashRt = (RectTransform)flashGo.transform;
    flashRt.anchorMin = flashRt.anchorMax = new Vector2(0.5f, 0.5f);
    flashRt.anchoredPosition = new Vector2(0, 120);
    flashRt.sizeDelta = new Vector2(900, 900);
    _sealFlash = flashGo.GetComponent<Image>();
    _sealFlash.sprite = _square;
    _sealFlash.color = new Color(PalGold.r, PalGold.g, PalGold.b, 0f);
    MakeText(_sealOverlay, "      ·      ", 72, new Vector2(0, -280), TextAnchor.MiddleCenter, 1000, 100).color = Pa
    _sealOverlay.gameObject.SetActive(false);
}

```

```
void BuildFailOverlay()
{
    _failOverlay = MakeOverlayPanel("FailOverlay", new Color(0.08f, 0.08f, 0.09f, 0.8f));
    MakeText(_failOverlay, "      ", 84, new Vector2(0, 200), TextAnchor.MiddleCenter, 1000, 110).color = new Color(0.08f, 0.08f, 0.09f, 0.8f);
    MakeText(_failOverlay, "      —          x1      50% \n      " + NewbieConfig.PityGuaranteeCount + "      ", 46, new Vector2(0, 200), TextAnchor.MiddleCenter, 1000, 110).color = new Color(0.08f, 0.08f, 0.09f, 0.8f);
    _failOverlay.gameObject.SetActive(false);
}

void BuildSkipBanner()
{
    var go = new GameObject("SkipBanner", typeof(RectTransform), typeof(Image));
    go.transform.SetParent(_canvas.transform, false);
    _skipBanner = (RectTransform)go.transform;
    _skipBanner.anchorMin = new Vector2(0f, 1f);
    _skipBanner.anchorMax = new Vector2(1f, 1f);
    _skipBanner.pivot = new Vector2(0.5f, 1f);
    _skipBanner.anchoredPosition = Vector2.zero;
    _skipBanner.sizeDelta = new Vector2(0, 340);
    var img = go.GetComponent<Image>();
    img.sprite = _square;
    img.color = new Color(0.10f, 0.38f, 0.35f, 0.9f);
    MakeText(_skipBanner, "      ", 80, new Vector2(0, 190), TextAnchor.MiddleCenter, 1000, 100);
    MakeText(_skipBanner, " " + NewbieConfig.FloorSkipFrom + " → " + NewbieConfig.FloorSkipTo + " ", 40, new Vector2(0, 190), TextAnchor.MiddleCenter, 1000, 100);
    _skipBanner.gameObject.SetActive(false);
}

void BuildWallBanner()
{
    var go = new GameObject("WallBanner", typeof(RectTransform), typeof(Image));
    go.transform.SetParent(_canvas.transform, false);
    _wallBanner = (RectTransform)go.transform;
    _wallBanner.anchorMin = new Vector2(0f, 1f);
    _wallBanner.anchorMax = new Vector2(1f, 1f);
    _wallBanner.pivot = new Vector2(0.5f, 1f);
    _wallBanner.anchoredPosition = Vector2.zero;
    _wallBanner.sizeDelta = new Vector2(0, 420);
    var img = go.GetComponent<Image>();
    img.sprite = _square;
    img.color = new Color(0.45f, 0.11f, 0.13f, 0.88f);
    MakeText(_wallBanner, " 10 · ", 80, new Vector2(0, 130), TextAnchor.MiddleCenter, 1000, 100);
    MakeText(_wallBanner, " 10% —          ", 40, new Vector2(0, 20), TextAnchor.MiddleCenter, 1000, 70);
    var btnGo = new GameObject("HeavenAidButton", typeof(RectTransform), typeof(Image), typeof(Button));
    btnGo.transform.SetParent(_wallBanner, false);
    var btnRt = (RectTransform)btnGo.transform;
    btnRt.anchorMin = btnRt.anchorMax = new Vector2(0.5f, 0.5f);
    btnRt.anchoredPosition = new Vector2(0, -110);
    btnRt.sizeDelta = new Vector2(380, 100);
    var btnImg = btnGo.GetComponent<Image>();
    btnImg.sprite = _square;
    btnImg.color = new Color(0.35f, 0.35f, 0.35f, 0.8f);
    var btn = btnGo.GetComponent<Button>();
    btn.interactable = false;
```

```

        MakeText(btnGo.transform, "      ", 40, Vector2.zero, TextAnchor.MiddleCenter, 380, 100).color = new Color(
            _wallBanner.gameObject.SetActive(false));
    }
    void BuildFormationPanel()
    {
        _formationPanel = MakeOverlayPanel("FormationPanel", new Color(0.04f, 0.05f, 0.07f, 0.94f));
        MakeText(_formationPanel, "      ·      ", 72, new Vector2(0, 620), TextAnchor.MiddleCenter, 1000, 100).color = P
        MakeText(_formationPanel, "      ·      ", 42, new Vector2(0, 460), TextAnchor.MiddleCenter, 1000, 70);
        for (int i = 0; i < NewbieConfig.FriendCount; i++)
        {
            var slot = MakeSlot(_formationPanel, new Vector2(-300 + i * 300, 300), PalGreen);
            MakeText(slot, NewbieConfig.FriendNames[i].Substring(0, 1), 56, Vector2.zero, TextAnchor.MiddleCenter, 2
        }
        MakeText(_formationPanel, "      · 1      ", 42, new Vector2(0, 90), TextAnchor.MiddleCenter, 1000, 70).color = Pa
        var btnGo = new GameObject("PlaceGudiaoButton", typeof(RectTransform), typeof(Image), typeof(Button));
        btnGo.transform.SetParent(_formationPanel, false);
        var btnRt = (RectTransform)btnGo.transform;
        btnRt.anchorMin = btnRt.anchorMax = new Vector2(0.5f, 0.5f);
        btnRt.anchoredPosition = new Vector2(0, -80);
        btnRt.sizeDelta = new Vector2(300, 300);
        var btnImg = btnGo.GetComponent<Image>();
        btnImg.sprite = _square;
        btnImg.color = PalFrostedGold; //      ·      §
        MakeText(btnGo.transform, "      ", 80, new Vector2(0, 40), TextAnchor.MiddleCenter, 300, 200).color = PalBlack;
        MakeText(btnGo.transform, "      ", 36, new Vector2(0, -100), TextAnchor.MiddleCenter, 300, 80).color = Pal
        btnGo.GetComponent<Button>().onClick.AddListener(() =>{
            _formationPanel.gameObject.SetActive(false);
            FormationPlaced?.Invoke();
        });
        MakeText(_formationPanel, "      —      +18%", 36, new Vector2(0, -320), TextAnchor.MiddleCenter
        _formationPanel.gameObject.SetActive(false);
    }
    void BuildSettlePanel()
    {
        _settlePanel = MakeOverlayPanel("SettlePanel", new Color(0.04f, 0.05f, 0.07f, 0.96f));
        MakeText(_settlePanel, "      ·      ", 76, new Vector2(0, 520), TextAnchor.MiddleCenter, 1040, 110).color =
        _settleStats = MakeText(_settlePanel, "", 44, new Vector2(0, 280), TextAnchor.MiddleCenter, 1020, 340);
        MakeActionButton(_settlePanel, "      ", new Vector2(-260, -300), () => UnityEngine.SceneManagement.SceneMa
        MakeActionButton(_settlePanel, "      ", new Vector2(260, -300), () => UnityEngine.SceneManagement.SceneMa
        _settlePanel.gameObject.SetActive(false);
    }
    // ————— reconcile —————
    public void SyncView(IdleSim sim)
    {
        ReconcileEnemies(sim);
        foreach (var kv in _enemies)
        {
            var v = kv.Value;
            v.Rt.anchoredPosition = new Vector2(FriendX + v.Pos * FieldSpan, RowY[v.Row]);
        }
    }

```

```

        v.Fill.fillAmount = v.MaxHp > 0f ? v.Hp / v.MaxHp : 0f;
        if (v.SealReady != v.LastSealReady)
        {
            v.LastSealReady = v.SealReady;
            v.SealLabel.gameObject.SetActive(v.SealReady);
            v.Fill.color = v.SealReady ? PalGold : new Color(0.2f, 0.9f, 0.3f);
            v.LastAttempts = -1;
        }
        if (v.SealReady && v.LastAttempts != v.Attempts)
        {
            v.LastAttempts = v.Attempts;
            int next = v.Attempts + 1;
            v.SealLabel.text = next >= NewbieConfig.PityGuaranteeCount
                ? "    " + next + "/" + NewbieConfig.PityGuaranteeCount + " . "
                : "    " + next + "/" + NewbieConfig.PityGuaranteeCount + " ";
        }
        if (v.Shielded != v.LastShielded)
        {
            v.LastShielded = v.Shielded;
            v.ShieldLabel.gameObject.SetActive(v.Shielded);
        }
    }
    for (int i = 0; i < _friendHpFills.Length; i++)
    {
        var f = sim.Friends[i];
        _friendHpFills[i].fillAmount = f.MaxHp > 0f ? f.Hp / f.MaxHp : 0f;
    }
    bool canSeal = sim.SealReadyEnemy != null;
    if (_sealButton.interactable != canSeal)
    {
        _sealButton.interactable = canSeal;
        _sealButtonImage.color = canSeal ? PalGold : new Color(0.35f, 0.35f, 0.38f);
        _sealLabel.color = canSeal ? PalBlack : new Color(0.7f, 0.7f, 0.7f);
    }
}

void ReconcileEnemies(IdleSim sim)
{
    _scratch.Clear();
    foreach (var kv in _enemies)
        if (!ContainsEnemy(sim, kv.Key)) _scratch.Add(kv.Key);
    for (int i = 0; i < _scratch.Count; i++)
    {
        Destroy(_enemies[_scratch[i]].Go);
        _enemies.Remove(_scratch[i]);
    }
    for (int i = 0; i < sim.Enemies.Count; i++)
    {
        var e = sim.Enemies[i];
        if (!_enemies.TryGetValue(e.Id, out var v))
        {

```

```

        v = CreateEnemyVisual(e);
        _enemies[e.Id] = v;
    }
    v.Pos = e.Pos;
    v.Row = e.Row;
    v.Hp = e.Hp;
    v.MaxHp = e.MaxHp;
    v.SealReady = e.SealReady;
    v.Shielded = e.Shielded;
    v.Attempts = e.SealAttempts;
}
}
static bool ContainsEnemy(Idlesim sim, int id)
{
    for (int i = 0; i < sim.Enemies.Count; i++)
        if (sim.Enemies[i].Id == id) return true;
    return false;
}
EnemyVisual CreateEnemyVisual(Idlesim.EnemyInst e)
{
    var go = new GameObject("Enemy_" + e.Id, typeof(RectTransform), typeof(Image));
    go.transform.SetParent(_field, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.sizeDelta = e.Boss ? new Vector2(200, 200) : e.Collectible ? new Vector2(160, 160) : new Vector2(90, 90);
    var image = go.GetComponent<Image>();
    var bossArt = e.Boss ? Sy.Core.ArtRuntime.Get("P04_unit_01") : null;
    if (bossArt != null)
    {
        image.sprite = bossArt;
        image.color = Color.white;
        image.preserveAspect = true;
    }
    else
    {
        image.sprite = _square;
        image.color = e.Boss ? PalBlue : e.Collectible ? PalGold : PalRed; //
    }
}
if (e.Collectible)
    MakeText(go.transform, e.Name.Substring(0, 1), 64, Vector2.zero, TextAnchor.MiddleCenter, 200, 200).color
var hpGo = new GameObject("Hp", typeof(RectTransform), typeof(Image));
hpGo.transform.SetParent(go.transform, false);
var hpRt = (RectTransform)hpGo.transform;
hpRt.anchorMin = new Vector2(0f, 1f);
hpRt.anchorMax = new Vector2(1f, 1f);
hpRt.anchoredPosition = new Vector2(0, -8);
hpRt.sizeDelta = new Vector2(-10, 12);
var hp = hpGo.GetComponent<Image>();
hp.sprite = _square;
hp.type = Image.Type.Filled;

```

```

        hp.fillMethod = Image.FillMethod.Horizontal;
        hp.color = new Color(0.2f, 0.9f, 0.3f);
        var seallabel = MakeText(go.transform, "    ", 34, new Vector2(0, 150), TextAnchor.MiddleCenter, 300, 50);
        seallabel.color = PalGold;
        seallabel.gameObject.SetActive(false);
        var shieldLabel = MakeText(go.transform, "    ", 34, new Vector2(0, 190), TextAnchor.MiddleCenter, 280, 50);
        shieldLabel.color = PalBlue;
        shieldLabel.gameObject.SetActive(false);
        return new EnemyVisual
        {
            Go = go,
            Rt = rt,
            Fill = hp,
            Seallabel = seallabel,
            ShieldLabel = shieldLabel.gameObject,
            Pos = e.Pos,
            Row = e.Row,
            Hp = e.Hp,
            MaxHp = e.MaxHp,
        };
    }

    // —— HUD ——

    public void SetFloor(int f) => _floorText.text = " " + f + " ";
    public void SetCharms(int n) => _charmText.text = "    x" + n;
    public void SetKills(int k) => _killsText.text = "    " + k;
    public void SetHint(string hint) => _hintText.text = hint;
    public void HideIntro() => _introOverlay.gameObject.SetActive(false);
    public void ShowWallBanner(bool show) => _wallBanner.gameObject.SetActive(show);
    public void ShowFormationPanel(bool show) => _formationPanel.gameObject.SetActive(show);
    public void ShowFloorSkip(bool show) => _skipBanner.gameObject.SetActive(show);
    public void SetCodex(int litCount)
    {
        for (int i = 0; i < NewbieConfig.CodexSlots; i++)
        {
            bool lit = i < litCount;
            _codexSlots[i].color = lit ? PalFrostedGold : new Color(0.16f, 0.16f, 0.18f); // .
            _codexLabels[i].text = lit ? (i == 0 ? " " : " ") : "?";
            _codexLabels[i].color = lit ? PalBlack : new Color(0.45f, 0.45f, 0.48f);
        }
    }

    public void PlaySealAnim()
    {
        StartCoroutine(SealAnimRoutine());
    }

    IEnumerator SealAnimRoutine()
    {
        _sealOverlay.gameObject.SetActive(true);
        float t = 0f;
        bool flashed = false;
        while (t < NewbieConfig.SealAnimSeconds)

```

```

    {
        t += Time.unscaledDeltaTime;
        float k = t / NewbieConfig.SealAnimSeconds;
        _magicRing.localRotation = Quaternion.Euler(0f, 0f, k * 540f);           // 1.5
        _magicRing.localScale = Vector3.one * Mathf.Lerp(1.3f, 0.55f, k);       //
        if (!flashed && k > 0.75f)
        {
            flashed = true;
            StartCoroutine(FlashRoutine());
        }
        yield return null;
    }
    _sealOverlay.gameObject.SetActive(false);
    _magicRing.localRotation = Quaternion.identity;
    _magicRing.localScale = Vector3.one;
}
IEnumerator FlashRoutine()
{
    float t = 0f;
    while (t < 0.25f)
    {
        t += Time.unscaledDeltaTime;
        float a = Mathf.PingPong(t / 0.25f * 2f, 1f);
        _sealFlash.color = new Color(PalGold.r, PalGold.g, PalGold.b, 0.75f * a);
        yield return null;
    }
    _sealFlash.color = new Color(PalGold.r, PalGold.g, PalGold.b, 0f);
}
public void PlaySealFail()
{
    StartCoroutine(FailRoutine());
}
IEnumerator FailRoutine()
{
    _failOverlay.gameObject.SetActive(true);
    yield return new WaitForSecondsRealtime(NewbieConfig.SealFailAnimSeconds);
    _failOverlay.gameObject.SetActive(false);
}
public void ShowSettle(int kills, int charms, int codex)
{
    _settlePanel.gameObject.SetActive(true);
    _settleStats.text = $"          {codex}/{NewbieConfig.CodexSlots}·      \n      ×{charms}      +{Newbie
}
// ——
RectTransform MakeOverlayPanel(string name, Color color)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = Vector2.zero;

```

```
        rt.anchorMax = Vector2.one;
        rt.offsetMin = rt.offsetMax = Vector2.zero;
        var image = go.GetComponent<Image>();
        image.sprite = _square;
        image.color = color;
        return rt;
    }
}

RectTransform MakeSlot(RectTransform parent, Vector2 pos, Color color)
{
    var go = new GameObject("Slot", typeof(RectTransform), typeof(Image));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = new Vector2(220, 220);
    var image = go.GetComponent<Image>();
    image.sprite = _square;
    image.color = new Color(color.r, color.g, color.b, 0.6f);
    return rt;
}

void MakeActionButton(RectTransform parent, string text, Vector2 pos, Action onClick)
{
    var go = new GameObject(text, typeof(RectTransform), typeof(Image), typeof(Button));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = new Vector2(420, 120);
    var image = go.GetComponent<Image>();
    image.sprite = _square;
    image.color = PalGreen;
    MakeText(go.transform, text, 46, Vector2.zero, TextAnchor.MiddleCenter, 420, 120);
    go.GetComponent<Button>().onClick.AddListener(() => onClick());
}

static Text MakeText(Transform parent, string text, int size, Vector2 pos, TextAnchor anchor, float w, float h)
{
    var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = new Vector2(w, h);
    var t = go.GetComponent<Text>();
    t.font = Resources.GetBuiltinResource<Font>("LegacyRuntime.ttf");
    t.text = text;
    t.fontSize = size;
    t.alignment = anchor;
    t.color = new Color(0.91f, 0.89f, 0.85f);
    t.supportRichText = true;
    return t;
}
```

```

    }
    static Sprite MakeSquareSprite()
    {
        var tex = new Texture2D(8, 8, TextureFormat.RGBA32, false);
        var px = tex.GetPixels32();
        for (int i = 0; i < px.Length; i++) px[i] = new Color32(255, 255, 255, 255);
        tex.SetPixels32(px);
        tex.Apply(false, false);
        var sprite = Sprite.Create(tex, new Rect(0, 0, 8, 8), new Vector2(0.5f, 0.5f), 8f);
        return sprite;
    }
    // -----
    sealed class EnemyVisual
    {
        public GameObject Go;
        public RectTransform Rt;
        public Image Fill;
        public Text Seallabel;
        public GameObject ShieldLabel;
        public bool LastSealReady;
        public bool LastShielded;
        public int LastAttempts;
        public float Pos;
        public int Row;
        public float Hp;
        public float MaxHp;
        public bool SealReady;
        public bool Shielded;
        public int Attempts;
    }
}
// IdleSim.cs
using System;
using System.Collections.Generic;
namespace Sy.P04.Newbie
{
    /// <summary>
    /// C#      Play  Editor  headless  NewbieFlowCheck
    /// X025    .    V1.2
    /// ①      1-3  →    4-9  → 10    —$4.1  "    "
    /// ②      $4.2-1    50% + 3      —      x2→
    /// ③      $4.3 IdleFormula  2/    —
    ///      HP    +    + Buff  10%
    /// </summary>
    public sealed class IdleSim
    {
        public enum IdleState { Prep, Fighting, SealAnim, SealFailAnim, FloorSkip, WallDemo, Formation, BossFight, Sett1
        public sealed class FriendInst
        {

```

```
        public int Id;
        public string Name;
        public int Row;
        public float Hp;
        public float MaxHp;
        public float Cooldown;
    }
    public sealed class EnemyInst
    {
        public int Id;
        public string Name;
        public int Row;
        public float Pos;          // 1.0=    → FriendEngageRange=
        public float Hp;
        public float MaxHp;
        public float Atk;
        public float Speed;
        public float Cooldown;
        public bool Collectible;
        public bool Boss;
        public bool Shielded;
        public bool SealReady;
        public bool Dead;
        public int SealAttempts;    //          3
    }
    public IdleState State { get; private set; } = IdleState.Prep;
    readonly List<FriendInst> _friends = new();
    readonly List<EnemyInst> _enemies = new();
    readonly List<EnemyInst> _deadThisTick = new();
    int _nextId = 1;
    int _floor;          // 1..3  =10
    int _floorSpawnIdx;
    float _floorTime;    //
    float _floorGapTimer; //          >0=
    float _prepTimer;
    float _sealTimer;
    float _wallTimer;
    float _skipTimer;
    bool _firstSealDone;
    EnemyInst _sealTarget;
    EnemyInst _sealReadyEnemy;
    int _attackRotate;
    public IReadOnlyList<FriendInst> Friends => _friends;
    public IReadOnlyList<EnemyInst> Enemies => _enemies;
    public int Life => NewbieConfig.StartLife;
    public int SealCharms { get; private set; } = NewbieConfig.StartSealCharms;
    public int CodexLitCount { get; private set; }
    public int Kills { get; private set; }
    public int Seals { get; private set; } //
    public int SealFails { get; private set; } //
```

```

public int SealRefunds { get; private set; } //
public int PitySaves { get; private set; } //
public int IdleIncomeGranted { get; private set; } //
public bool FloorSkipShown { get; private set; } // 4-9
public bool GudiaoJoined { get; private set; } //
public float BattleTime { get; private set; }
public float YinBuff { get; private set; } = 1f;
public EnemyInst SealReadyEnemy => _sealReadyEnemy;
public bool AdPopupShown => false; //
/// <summary> Prep=0 1-3 =9 =10 </summary>
public int Floor
{
    get
    {
        if (State == IdleState.Prep) return 0;
        if (State == IdleState.FloorSkip) return NewbieConfig.FloorSkipTo;
        if (State == IdleState.Settled || State == IdleState.Lost) return NewbieConfig.WallFloor;
        if (State == IdleState.WallDemo || State == IdleState.Formation || State == IdleState.BossFight
            || State == IdleState.SealAnim || State == IdleState.SealFailAnim)
            return _firstSealDone ? NewbieConfig.WallFloor : _floor;
        return _floor;
    }
}
public event Action<EnemyInst> EnemyKilled;
public event Action<EnemyInst> SealEnabled;
public event Action<EnemyInst> Sealed;
public event Action<EnemyInst, int> SealFailed; // ( , )
public event Action WallDemoShown;
public event Action FormationUnlocked;
public event Action GudiaoPlaced;
public event Action<int> CharmsChanged;
public event Action<IdleState> StateChanged;
public IdleSim()
{
    for (int i = 0; i < NewbieConfig.FriendCount; i++)
    {
        _friends.Add(new FriendInst
        {
            Id = _nextId++,
            Name = NewbieConfig.FriendNames[i],
            Row = i,
            Hp = NewbieConfig.FriendHp,
            MaxHp = NewbieConfig.FriendHp,
        });
    }
}
public void Tick(float dt)
{
    switch (State)
    {

```

```
        case IdleState.Prep:
            _prepTimer += dt;
            if (_prepTimer >= NewbieConfig.IntroSeconds) StartHunt();
            return;
        case IdleState.Fighting:
        case IdleState.BossFight:
            TickBattle(dt);
            return;
        case IdleState.SealAnim:
            _sealTimer -= dt;
            if (_sealTimer <= 0f) FinishSeal();
            return;
        case IdleState.SealFailAnim:
            _sealTimer -= dt;
            if (_sealTimer <= 0f) FinishSealFail();
            return;
        case IdleState.FloorSkip:
            BattleTime += dt;
            _skipTimer -= dt;
            if (_skipTimer <= 0f) EnterWallDemo();
            return;
        case IdleState.WallDemo:
            BattleTime += dt;
            TickEnemiesMove(dt);
            _wallTimer -= dt;
            if (_wallTimer <= 0f)
            {
                SetState(IdleState.Formation);
                FormationUnlocked?.Invoke();
            }
            return;
        // Formation
    }
}

void StartHunt()
{
    if (State != IdleState.Prep) return;
    BattleTime = 0f;
    BeginFloor(1);
}

void BeginFloor(int floor)
{
    _floor = floor;
    _floorSpawnIdx = 0;
    _floorTime = 0f;
    _floorGapTimer = 0f;
    SetState(IdleState.Fighting);
}

SpawnDef[] CurrentFloorSpawns() => NewbieConfig.FloorSpawns[_floor - 1];
void TickBattle(float dt)
```

```

{
    BattleTime += dt;
    if (State == IdleState.Fighting)
    {
        if (_floorGapTimer > 0f)
        {
            _floorGapTimer -= dt;
            if (_floorGapTimer <= 0f) BeginFloor(_floor + 1);
            return;
        }
        _floorTime += dt;
        TickFloorSpawns();
        //          +      3      — HP      FinishSeal
        if (_floorSpawnIdx >= CurrentFloorSpawns().Length && _enemies.Count == 0
            && _floor < NewbieConfig.TutorialFloors)
        {
            _floorGapTimer = NewbieConfig.FloorGapSeconds;
        }
    }
    TickEnemiesMove(dt);
    TickFriendsAttack(dt);
    TickEnemiesAttack(dt);
    RemoveDead();
}

void TickFloorSpawns()
{
    var spawns = CurrentFloorSpawns();
    while (_floorSpawnIdx < spawns.Length && spawns[_floorSpawnIdx].At <= _floorTime)
    {
        _enemies.Add(NewEnemy(spawns[_floorSpawnIdx], shielded: false));
        _floorSpawnIdx++;
    }
}

EnemyInst NewEnemy(SpawnDef def, bool shielded)
{
    return new EnemyInst
    {
        Id = _nextId++,
        Name = def.Name,
        Row = def.Row,
        Pos = 1f,
        Hp = def.Hp,
        MaxHp = def.Hp,
        Atk = def.Atk,
        Speed = def.Speed,
        Cooldown = def.Atk > 0f ? 1f : 0f,
        Collectible = def.Collectible,
        Boss = def.Boss,
        Shielded = shielded,
    };
}

```

```
}
void TickEnemiesMove(float dt)
{
    for (int i = 0; i < _enemies.Count; i++)
    {
        var e = _enemies[i];
        if (e.Pos > NewbieConfig.FriendEngageRange)
            e.Pos = Math.Max(NewbieConfig.FriendEngageRange, e.Pos - e.Speed * dt);
    }
}
void TickFriendsAttack(float dt)
{
    for (int i = 0; i < _friends.Count; i++)
    {
        var f = _friends[i];
        f.Cooldown -= dt;
        if (f.Cooldown > 0f) continue;
        var target = PickTarget(f);
        if (target == null) { f.Cooldown = 0.05f; continue; }
        f.Cooldown = NewbieConfig.FriendAtkInterval;
        DamageEnemy(target, NewbieConfig.FriendAtk * YinBuff);
    }
}
EnemyInst PickTarget(FriendInst f)
{
    // —
    EnemyInst best = null;
    for (int i = 0; i < _enemies.Count; i++)
    {
        var e = _enemies[i];
        if (e.Dead || e.Pos > NewbieConfig.FriendEngageRange) continue;
        if (e.Row == f.Row) { best = e; break; }
        if (best == null) best = e;
    }
    return best;
}
void DamageEnemy(EnemyInst e, float dmg)
{
    if (e.Shielded) return;
    float lockHp = e.Collectible ? e.MaxHp * NewbieConfig.SealHpRatio : 0f;
    e.Hp -= dmg;
    if (e.Hp > lockHp) return;
    if (!e.Collectible)
    {
        e.Dead = true;
        _deadThisTick.Add(e);
        return;
    }
    e.Hp = lockHp; // HP
    if (e.SealReady) return;
```

```

        e.SealReady = true;
        _sealReadyEnemy = e;
        SealEnabled?.Invoke(e);
    }
    void TickEnemiesAttack(float dt)
    {
        for (int i = 0; i < _enemies.Count; i++)
        {
            var e = _enemies[i];
            if (e.Atk <= 0f || e.Dead || e.Shielded) continue;
            if (e.Pos > NewbieConfig.FriendEngageRange) continue;
            e.Cooldown -= dt;
            if (e.Cooldown > 0f) continue;
            e.Cooldown = 1.2f;
            var f = _friends[_attackRotate % _friends.Count];
            _attackRotate++;
            f.Hp = Math.Max(1f, f.Hp - e.Atk); // 5 +
        }
    }
    void RemoveDead()
    {
        for (int i = 0; i < _deadThisTick.Count; i++)
        {
            var e = _deadThisTick[i];
            _enemies.Remove(e);
            Kills++;
            EnemyKilled?.Invoke(e);
        }
        _deadThisTick.Clear();
    }
    /// <summary> HP 2 §4.2-1
    /// 06 M2 10 = x2→ 3 </summary>
    public bool PressSeal()
    {
        if (_sealReadyEnemy == null || _sealReadyEnemy.Dead) return false;
        if (State != IdleState.Fighting && State != IdleState.BossFight) return false;
        if (SealCharms < NewbieConfig.SealCost) return false;
        _sealTarget = _sealReadyEnemy;
        _sealTarget.SealAttempts++;
        SealCharms -= NewbieConfig.SealCost;
        CharmsChanged?.Invoke(SealCharms);
        bool success = _sealTarget.Boss
            ? _sealTarget.SealAttempts >= NewbieConfig.PityGuaranteeCount
            : true;
        _sealTimer = success ? NewbieConfig.SealAnimSeconds : NewbieConfig.SealFailAnimSeconds;
        SetState(success ? IdleState.SealAnim : IdleState.SealFailAnim);
        return true;
    }
    void FinishSeal()
    {

```

```

    var e = _sealTarget;
    _sealTarget = null;
    _sealReadyEnemy = null;
    e.Dead = true;
    _enemies.Remove(e);
    Seals++;
    if (e.Boss && e.SealAttempts >= NewbieConfig.PityGuaranteeCount) PitySaves++;
    CodexLitCount++;
    Sealed?.Invoke(e);
    if (!_firstSealDone)
    {
        _firstSealDone = true;
        SetState(IdleState.FloorSkip);
        _skipTimer = NewbieConfig.FloorSkipSeconds;
        FloorSkipShown = true;
    }
    else
    {
        //          §4.3 IdleFormula      2/
        IdleIncomeGranted = NewbieConfig.IdleIncomeCharmsPerDay;
        SealCharms += IdleIncomeGranted;
        CharmsChanged?.Invoke(SealCharms);
        SetState(IdleState.Settled);
    }
}

void FinishSealFail()
{
    var e = _sealTarget;
    _sealTarget = null;
    SealFails++;
    SealCharms += NewbieConfig.FailRefundCharm;    //    ①    50%
    SealRefunds++;
    CharmsChanged?.Invoke(SealCharms);
    SealFailed?.Invoke(e, e.SealAttempts);
    SetState(IdleState.BossFight);                //    HP    —
}

void EnterWallDemo()
{
    _floor = NewbieConfig.WallFloor;
    SetState(IdleState.WallDemo);
    _wallTimer = NewbieConfig.WallDemoSeconds;
    _enemies.Add(NewEnemy(NewbieConfig.WallBoss, shielded: true));
    WallDemoShown?.Invoke();
}

/// <summary>M4 — +18% 10% </summary>
public bool PlaceGudiao()
{
    if (State != IdleState.Formation) return false;
    YinBuff = 1f + NewbieConfig.YinSlotAtkBuff;
    GudiaoJoined = true;
}

```

```

        for (int i = 0; i < _enemies.Count; i++) _enemies[i].Shielded = false;
        GudiaoPlaced?.Invoke();
        SetState(IdleState.BossFight);
        return true;
    }
    void SetState(IdleState s)
    {
        if (State == s) return;
        State = s;
        StateChanged?.Invoke(s);
    }
}
}
// NewbieConfig.cs
namespace Sy.P04.Newbie
{
    public readonly struct SpawnDef
    {
        public readonly float At;          //
        public readonly string Name;
        public readonly int Row;          //   lane 0..2
        public readonly float Hp;
        public readonly float Atk;        // 0=
        public readonly float Speed;      //   /
        public readonly bool Collectible; //   HP +
        public readonly bool Boss;
        public SpawnDef(float at, string name, int row, float hp, float atk, float speed, bool collectible, bool boss)
        {
            At = at;
            Name = name;
            Row = row;
            Hp = hp;
            Atk = atk;
            Speed = speed;
            Collectible = collectible;
            Boss = boss;
        }
    }
}
/// <summary>
/// P04      V0.2 X025   .   "   " —           X009+IConfigService      Design/configs/P04 JSON
///      V1.2 —$4.1           →   →   →           $4.2-1           50% +   3
/// $4.3   +IdleFormula sealCharmBasePerDay:2   $4.4   10-12   10% $   =           0.8s+           /
///      #E63946 / #457B9D / #FFD700 / #2A9D8F / #1D1D1D
/// </summary>
public static class NewbieConfig
{
    public const string GameTitle = "   ";
    public const string GameSlogan = "   .   ";
    //   $4.2-2           06 M4 = 3+ 1
    public const int FriendCount = 3;
}

```

```

public const float FriendHp = 100f;
public const float FriendAtk = 6f;
public const float FriendAtkInterval = 0.8f;
public const float FriendEngageRange = 0.3f;
public const int StartLife = 3; //
// §4.3 IdleFormula 2/ —
public const int StartSealCharms = 9;
public const int SealCost = 2; // 2
public const int FailRefundCharm = 1; // ① 50% 2 → 1 §4.2-1
public const int PityGuaranteeCount = 3; // ② 3 " → "
public const int IdleIncomeCharmsPerDay = 2; // §4.3
public const int CodexSlots = 2; // +
// §4.1 " " §4.4 10
public const int TutorialFloors = 3; // 1-3
public const int WallFloor = 10; // = 10
public const int FloorSkipFrom = 4; //
public const int FloorSkipTo = 9;
public const float FloorSkipSeconds = 2.5f;
public const float FloorGapSeconds = 1.5f;
public const float SpawnWalkSpeed = 0.55f;
public const float IntroSeconds = 2.0f;
public const float SealHpRatio = 0.2f; // HP 20%
public const float SealAnimSeconds = 0.8f; // 0.8s § V1.2
public const float SealFailAnimSeconds = 0.5f; // .
public const float WallGapRatio = 0.10f; // 10 10%
public const float WallDemoSeconds = 8f;
public const float YinSlotAtkBuff = 0.18f; // +18% 10% =
public const float MaxTutorialSeconds = 210f; // ≤3.5min 06 ②
public static readonly string[] FriendNames = { " ", " ", " " };
/// &lt;summary&gt; 1-3 3 06 M2 &lt;/summary&gt;
public static readonly SpawnDef[][] FloorSpawns =
{
    new[] // 1
    {
        new SpawnDef(0.5f, " ", 0, 18f, 0f, SpawnWalkSpeed, false, false),
        new SpawnDef(1.0f, " ", 1, 18f, 0f, SpawnWalkSpeed, false, false),
        new SpawnDef(1.5f, " ", 2, 18f, 0f, SpawnWalkSpeed, false, false),
    },
    new[] // 2
    {
        new SpawnDef(0.5f, " ", 0, 22f, 0f, SpawnWalkSpeed, false, false),
        new SpawnDef(1.0f, " ", 1, 22f, 0f, SpawnWalkSpeed, false, false),
        new SpawnDef(1.5f, " ", 2, 22f, 0f, SpawnWalkSpeed, false, false),
        new SpawnDef(2.0f, " ", 0, 22f, 0f, SpawnWalkSpeed, false, false),
    },
    new[] // 3 =
    {
        new SpawnDef(0.5f, " ", 0, 26f, 0f, SpawnWalkSpeed, false, false),
        new SpawnDef(1.0f, " ", 1, 26f, 0f, SpawnWalkSpeed, false, false),
        new SpawnDef(1.5f, " ", 2, 26f, 0f, SpawnWalkSpeed, false, false),
    }
}

```

```

        new SpawnDef(2.0f, " ", 1, 26f, 0f, SpawnWalkSpeed, false, false),
        new SpawnDef(3.0f, " ", 1, 120f, 6f, SpawnWalkSpeed, true, false),
    },
};
/// <summary> 10      Boss      HP 20%      </summary>
public static SpawnDef WallBoss => new SpawnDef(0f, " ", 1, 300f, 5f, SpawnWalkSpeed, true, true);
}
}
// TutorialFlow.cs
using System;
namespace Sy.P04.Newbie
{
    /// <summary>
    ///     evt_newbie_step 1-5 ① ② ③ ④ ⑤ +
    ///     IdleSim HP / + 3 / / Buff / / —
    /// Play headless NewbieFlowCheck
    /// </summary>
    public sealed class TutorialFlow
    {
        public int Step { get; private set; }
        readonly Action<string> _hint;
        readonly Action<int> _onStep;
        bool _firstKillSeen;
        bool _firstSealSeen;
        public TutorialFlow(IdleSim sim, Action<string> hint, Action<int> onStep)
        {
            _hint = hint ?? delegate { };
            _onStep = onStep ?? delegate { };
            sim.EnemyKilled += OnEnemyKilled;
            sim.SealEnabled += OnSealEnabled;
            sim.Sealed += OnSealed;
            sim.SealFailed += OnSealFailed;
            sim.WallDemoShown += OnWallDemoShown;
            sim.GudiaoPlaced += OnGudiaoPlaced;
            sim.StateChanged += OnStateChanged;
            _hint(" . — ");
        }
        void Advance(int step)
        {
            if (Step >= step) return;
            Step = step;
            _onStep(step);
        }
        void OnEnemyKilled(IdleSim.EnemyInst e)
        {
            if (_firstKillSeen) return;
            _firstKillSeen = true;
            Advance(1);
            _hint(" — ");
        }
    }
}

```

```
void OnSealEnabled(IdleSim.EnemyInst e)
{
    _hint(e.Name + "          —          ");
}
void OnSealed(IdleSim.EnemyInst e)
{
    if (!_firstSealSeen)
    {
        _firstSealSeen = true;
        Advance(2);
        _hint("          —" + e.Name + "          ");
    }
    else
    {
        _hint("      " + e.Name + "      —      ");
    }
}
void OnSealFailed(IdleSim.EnemyInst e, int attempts)
{
    if (attempts + 1 >= NewbieConfig.PityGuaranteeCount)
        _hint("      · 1          50% —      " + NewbieConfig.PityGuaranteeCount + "          ");
    else
        _hint("      · 1      —      " + NewbieConfig.PityGuaranteeCount + "          ");
}
void OnWallDemoShown()
{
    Advance(3);
    _hint(" 10          10%          —          ");
}
void OnGudiaoPlaced()
{
    Advance(4);
    _hint("          —      +18%          ");
}
void OnStateChanged(IdleSim.IdleState state)
{
    switch (state)
    {
        case IdleSim.IdleState.FloorSkip:
            _hint("          " + NewbieConfig.FloorSkipFrom + " → " + NewbieConfig.FloorSkipTo + " —          ");
            break;
        case IdleSim.IdleState.Formation:
            _hint("      —          ");
            break;
        case IdleSim.IdleState.Settled:
            Advance(5);
            _hint("      2 +      2 /      — 300          ");
            break;
    }
}
```

```
    }  
}
```